24CY764 Secure System Engineering
Lab Experiment Assignment-1

Name: KRISHNAPRIYA S

RegNo: CB.SC.P2CYS25013

Repository Link:

GitHub Repository: https://github.com/krishnapriyasvarma8/secure_login_cys25013

This repository contains all the source code for the secure login system implemented across the three tasks.

1.Secure Login with bcrypt

Build a Node.js or Flask login page that hashes passwords with bcrypt. Never store plaintext. Show hashed output in DB.

https://www.npmjs.com/package/bcrypt

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Submit GitHub repo with working login/register. Screenshot the database showing hashed passwords.

Secure Login with bcrypt:

A secure login and registration system was built using Python and Flask. The goal is to ensure that passwords are never stored in plaintext. Instead, they are hashed using the bcrypt algorithm before being saved to the database.

Tools and Technologies used:

- **Language:** Python 3.11
- **Framework:** Flask
- **Password Hashing:** bcrypt
- **Database:** SQLite
- **Editor:** VS Code

Project Structure:

The project consists of two main Python files and a templates folder. The app.py file is the main server file that handles all the routes. The database.py file is responsible for creating the database and the users table. Inside the templates folder, there are two HTML files, register.html for the registration page and login.html for the login page.

How the Registration Works

1. User enters a username and password on the register page
2. The password is encoded to bytes
3. bcrypt generates a random salt and hashes the password
4. The hashed password is stored in the database
5. Plaintext password is never saved anywhere

Code used for hashing:

if bcrypt.checkpw(password, user['password'].encode('utf-8')):

How the Login Works

1. User enters username and password
2. The entered password is compared with the stored hash using bcrypt
3. If it matches, login is successful
4. If not, an error message is shown

Code used for comparing:

if bcrypt.checkpw(password, user['password'].encode('utf-8')):

How bcrypt Protects Passwords

- bcrypt never produces the same hash twice even for the same password because it adds a random **salt**

- The $2b$12$... in the hash means bcrypt with cost factor 12, which means it does 4096 rounds of hashing
- This makes brute force attacks extremely slow and difficult
- Even if the database is stolen, the attacker cannot read the actual passwords

Step-by-step working:

Step 1: Running the Application

The application is started by running the following command in the terminal:

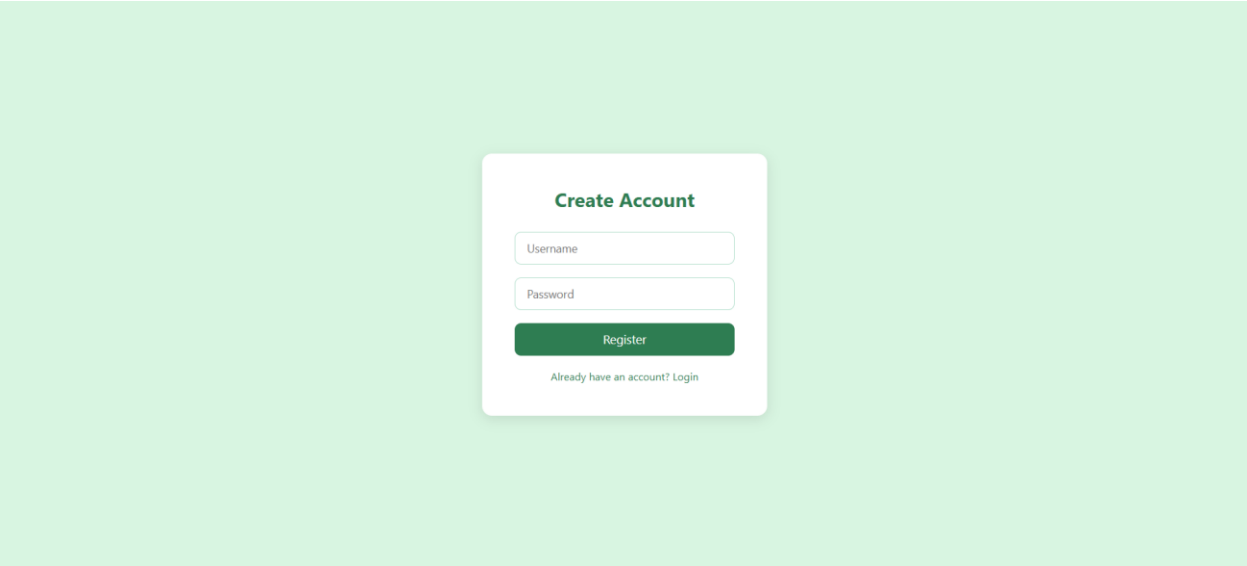
python app.py

Once running, the terminal displays a local server address. The application can then be accessed in the browser at http://127.0.0.1:5000/register and the login page can be accessed at http://127.0.0.1:5000/login

```
C:\Users\krish\OneDrive\Desktop\secure-login>python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 406-883-278
127.0.0.1 - - [28/Apr/2026 11:04:50] "GET /register HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 11:04:51] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [28/Apr/2026 11:06:48] "POST /register HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 11:06:48] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [28/Apr/2026 11:07:18] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 11:07:18] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [28/Apr/2026 11:08:09] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 11:08:09] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [28/Apr/2026 11:08:35] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 11:08:35] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [28/Apr/2026 11:08:35] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [28/Apr/2026 11:08:36] "GET /register HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 11:08:36] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [28/Apr/2026 11:10:27] "POST /register HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 11:10:28] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [28/Apr/2026 11:18:02] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 11:18:02] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [28/Apr/2026 11:18:14] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 11:18:14] "POST /predict HTTP/1.1" 404 -
 * Detected change in 'C:\\Users\\krish\\OneDrive\\Desktop\\secure-login\\app.py', reloading
 * Restarting with stat
 * Debugger is active!
```

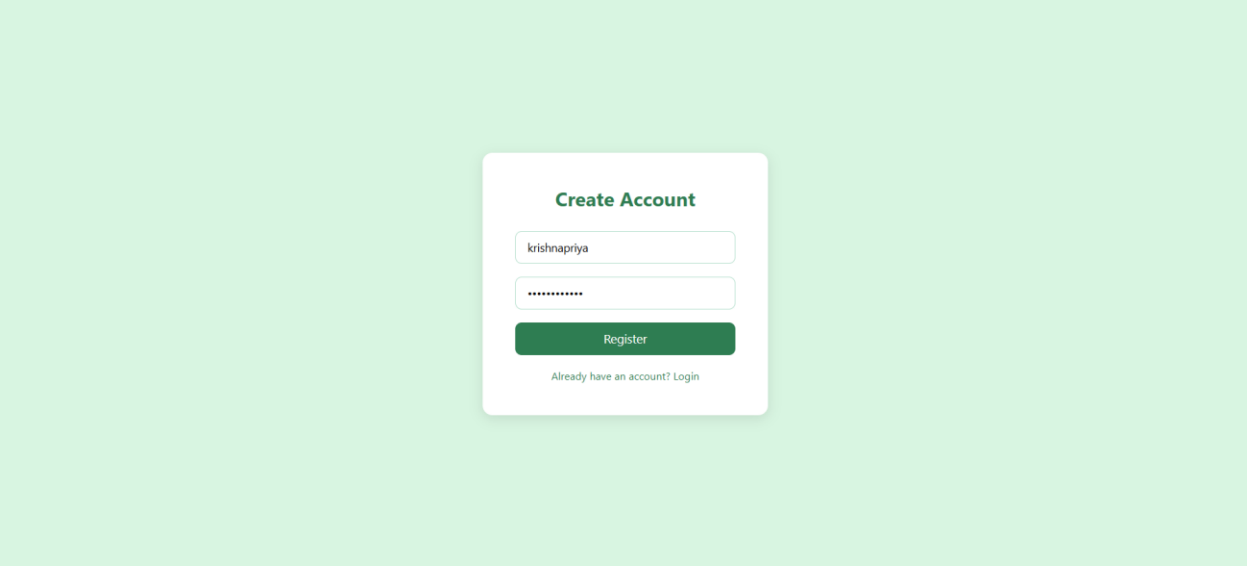
Step 2: Opening the Register Page

The user navigates to the register page where they are prompted to enter a username and password.



Step 3: Entering Credentials and Registering

The user enters a username and password and clicks the Register button.



Step 4: Password gets Hashed

Once registered, the system hashes the password using bcrypt and displays the hash on the screen. The plaintext password is never saved anywhere.

Registered successfully! Your password hash is : $2b$12$eEaWibRCaFypMgzz57OYtOmGzCBTMzNDu3.qTliGJ3kzolz9t/Ds2

Login here

Step 5: Hash Stored in Database

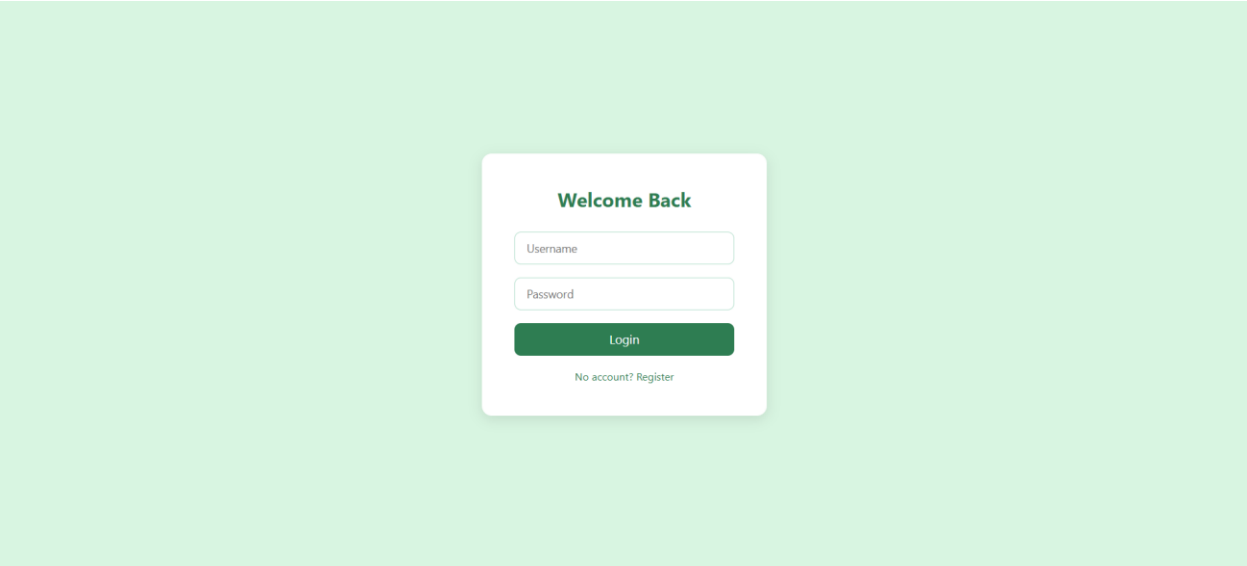
The hashed password is stored in the database. As seen below, the password column shows the bcrypt hash and not the actual password.

Rows: 6

	id	username	password
1	1	abc	$2b$12$RixlxbOi7kYtu5MHfMwKHunQ6dxFFxjXHm1./J...
2	2	kamal	$2b$12$vxJWV5OmPNJLj0I2FaKKp.RUZ1a3j4QQOnqU5...
3	3	noel	$2b$12$vCOE5ZBVWE/GgS8JbD9xiO7MK1HCslfOd7JeM...
4	4	pascal	$2b$12$VfIKJMF9r.z5HlQn8UDG8enzvCT9HtYsq0zUB1Q...
5	5	kp	$2b$12$.hPMNJGLLlqTTMyGthrWle/WyYbYR5YG5nxZd...
6	6	krishnapriya	$2b$12$eEaWibRCaFypMgzz57OYtOmGzCBTMzNDu3.q...
7			

Step 6: Opening the Login Page

The user navigates to the login page and enters their credentials.



Step 7: Login Successful

The system compares the entered password with the stored hash using bcrypt. If it matches, the user is granted access.

Login successful!

Step 8: Login Failed

If the user enters an incorrect password, bcrypt compares it with the stored hash, and if they do not match, the system displays an error message denying access.

Incorrect credentials

Conclusion

- The login system successfully hashes all passwords using bcrypt before storing.
- No plaintext passwords exist anywhere in the database.
- The system is protected against rainbow table attacks due to automatic salting.
- bcrypt's slow hashing makes it resistant to brute force attacks.

2. Using the created login page, simulate a credential stuffing attack. Then add rate limiting and account lockout to stop it.

https://github.com/dw0rsec/rockyou.txt

https://tryhackme.com/room/hydra

This task simulates a credential stuffing attack on the login system built in Task 1 using the rockyou.txt password wordlist. After demonstrating the attack, rate limiting and account lockout are implemented to defend against it.

Tools and Technologies Used

- Language: Python
- Framework: Flask
- Attack Wordlist: rockyou.txt
- Rate Limiting: flask-limiter
- HTTP Library: requests

Credential Stuffing Attack:

A credential stuffing attack is a cyberattack where an attacker automatically tries a large list of commonly used passwords against a login page. The rockyou.txt file, which contains over 14 million real world leaked passwords, is used here to simulate such an attack.

Rate Limiting:

Rate limiting restricts the number of login attempts within a given time period. Here, only 5 attempts per minute are allowed. Exceeding this limit results in a 429 Too Many Requests error.

Account Lockout:

Account lockout temporarily blocks an account after too many failed login attempts. In this implementation, an account is locked for 10 minutes after 5 consecutive failed attempts.

<u>Credential Stuffing attack using rockyou.txt:</u>

1. The users table shows 3 registered accounts with passwords secured using **bcrypt** hashing.

Rows: 3

	id	username	password
1	2	john	$2b$12$ue7cG6GOituAzoEPdZpi0eh.n7jg0jhZnz5Dn3W...
2	3	test	$2b$12$jfDyay6RQDZGrTwsiYKlou/iEl3EibNr6pjakjMT/8...
3	5	admin	$2b$12$CBWUB3kpay8z.Zj0cdmvcuAUx1jkh0uqcaxWrb...
4			

2. The **rockyou.txt** wordlist is downloaded and placed in the project directory alongside app.py, attack.py, and the database.

app		28-04-2026 11:18	Python Source File	2 KB
attack		28-04-2026 11:41	Python Source File	1 KB
database		27-04-2026 15:55	Python Source File	1 KB
rockyou		28-04-2026 11:35	Text Document	1,36,643 KB
users		28-04-2026 11:56	Data Base File	16 KB
users.db-x-users-1-password.bin		27-04-2026 16:12	BIN File	1 KB

Single account

app.py and attack_single.py are executed in parallel.

```
C:\Users\krish\OneDrive\Desktop\secure-login>python attack_single.py
Starting attack on account: admin
Trying 14344392 passwords...
----------------------------------------
Attempt 1: 123456 - Failed
Attempt 2: 12345 - Failed
Attempt 3: 123456789 - Failed
Attempt 4: password - Failed
Attempt 5: iloveyou - Failed
Attempt 6: princess - Failed
Attempt 7: 1234567 - Failed
Attempt 8: rockyou - Failed
Attempt 9: 12345678 - Failed
Attempt 10: abc123 - Failed
PASSWORD FOUND! Password is: nicole
Found after 11 attempts

C:\Users\krish\OneDrive\Desktop\secure-login>
```

```
C:\Users\krish\OneDrive\Desktop\secure-login>python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deploym
ent. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 406-883-278
127.0.0.1 - - [28/Apr/2026 13:28:11] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:28:11] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:28:11] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:28:11] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:28:12] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:28:12] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:28:12] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:28:12] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:28:13] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:28:13] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:28:13] "POST /login HTTP/1.1" 200 -
```

The Flask server processes incoming login requests while the attack script systematically submits passwords from the rockyou.txt wordlist, successfully retrieving admin's password "nicole" in just 11 attempts with no defensive mechanisms in place.

```
C:\Users\krish\OneDrive\Desktop\secure-login>python attack_single.py
Starting attack on account: admin
Trying 14344392 passwords...
----------------------------------------
Attempt 1: 123456 - Failed
Attempt 2: 12345 - Failed
Attempt 3: 123456789 - Failed
Attempt 4: password - Failed
Attempt 5: iloveyou - Failed
Attempt 6: princess - Failed
Attempt 7: 1234567 - Failed
Attempt 8: rockyou - Failed
Attempt 9: 12345678 - Failed
Attempt 10: abc123 - Failed
PASSWORD FOUND! Password is: nicole
Found after 11 attempts
```

Multiple accounts

attack_multiple.py extends the attack across all 3 accounts simultaneously, successfully compromising **admin** (11 attempts), **john** (4 attempts), and **test** (10 attempts), with every POST request returning 200, demonstrating that the unprotected server offers no resistance against automated credential stuffing at scale.

```
C:\Users\krish\OneDrive\Desktop\secure-login>python attack_multiple.py
Starting credential stuffing attack...
Trying 200 passwords on 3 accounts...
----------------------------------------

Attacking account: admin
Attempt 1: 123456 - Failed
Attempt 2: 12345 - Failed
Attempt 3: 123456789 - Failed
Attempt 4: password - Failed
Attempt 5: iloveyou - Failed
Attempt 6: princess - Failed
Attempt 7: 1234567 - Failed
Attempt 8: rockyou - Failed
Attempt 9: 12345678 - Failed
Attempt 10: abc123 - Failed
PASSWORD FOUND! admin:nicole (after 11 attempts)

Attacking account: john
Attempt 1: 123456 - Failed
Attempt 2: 12345 - Failed
Attempt 3: 123456789 - Failed
PASSWORD FOUND! john:password (after 4 attempts)

Attacking account: test
Attempt 1: 123456 - Failed
Attempt 2: 12345 - Failed
Attempt 3: 123456789 - Failed
Attempt 4: password - Failed
Attempt 5: iloveyou - Failed
Attempt 6: princess - Failed
Attempt 7: 1234567 - Failed
Attempt 8: rockyou - Failed
Attempt 9: 12345678 - Failed
PASSWORD FOUND! test:abc123 (after 10 attempts)

----------------------------------------
Attack complete! 3 accounts compromised:
  → admin:nicole
  → john:password
  → test:abc123

C:\Users\krish\OneDrive\Desktop\secure-login>
```

```
C:\Users\krish\OneDrive\Desktop\secure-login>python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deploym
ent. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 406-883-278
127.0.0.1 - - [28/Apr/2026 13:31:17] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:17] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:17] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:18] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:18] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:18] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:18] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:19] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:19] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:19] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:19] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:19] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:20] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:20] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:20] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:20] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:21] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:21] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:21] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:21] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:22] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:22] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:22] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:22] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 13:31:22] "POST /login HTTP/1.1" 200 -
```

```
C:\Users\krish\OneDrive\Desktop\secure-login>python attack_multiple.py
Starting credential stuffing attack...
Trying 200 passwords on 3 accounts...
----------------------------------------

Attacking account: admin
Attempt 1: 123456 - Failed
Attempt 2: 12345 - Failed
Attempt 3: 123456789 - Failed
Attempt 4: password - Failed
Attempt 5: iloveyou - Failed
Attempt 6: princess - Failed
Attempt 7: 1234567 - Failed
Attempt 8: rockyou - Failed
Attempt 9: 12345678 - Failed
Attempt 10: abc123 - Failed
PASSWORD FOUND! admin:nicole (after 11 attempts)

Attacking account: john
Attempt 1: 123456 - Failed
Attempt 2: 12345 - Failed
Attempt 3: 123456789 - Failed
PASSWORD FOUND! john:password (after 4 attempts)

Attacking account: test
Attempt 1: 123456 - Failed
Attempt 2: 12345 - Failed
Attempt 3: 123456789 - Failed
Attempt 4: password - Failed
Attempt 5: iloveyou - Failed
Attempt 6: princess - Failed
Attempt 7: 1234567 - Failed
Attempt 8: rockyou - Failed
Attempt 9: 12345678 - Failed
PASSWORD FOUND! test:abc123 (after 10 attempts)

----------------------------------------
Attack complete! 3 accounts compromised:
  → admin:nicole
  → john:password
  → test:abc123
```

Implementing Rate Limiting and Account Lockout:

With rate limiting and account lockout enforced, the server begins returning 429 (Too Many Requests) responses after a threshold is exceeded, blocking further attempts. Despite exhausting all 200 passwords across all 3 accounts, 0 accounts were compromised, confirming the defensive mechanisms successfully mitigated the credential stuffing attack.

```
C:\Users\krish\OneDrive\Desktop\secure-login>python attack_multiple.py
Starting credential stuffing attack...
Trying 200 passwords on 3 accounts...
--------------------------------------

Attacking account: admin
Attempt 1: 123456 - Failed
Attempt 2: 12345 - Failed
Attempt 3: 123456789 - Failed
Attempt 4: password - Failed
Attempt 5: iloveyou - Failed
Attempt 6: princess - Failed
Attempt 7: 1234567 - Failed
Attempt 8: rockyou - Failed
Attempt 9: 12345678 - Failed
Attempt 10: abc123 - Failed
Attempt 11: nicole - Failed
Attempt 12: daniel - Failed
Attempt 13: babygirl - Failed
Attempt 14: monkey - Failed
Attempt 15: lovely - Failed
Attempt 16: jessica - Failed
Attempt 17: 654321 - Failed
Attempt 18: michael - Failed
Attempt 19: ashley - Failed
Attempt 20: qwerty - Failed
Attempt 21: 111111 - Failed
Attempt 22: iloveu - Failed
Attempt 23: 000000 - Failed
Attempt 24: michelle - Failed
Attempt 25: tigger - Failed
Attempt 26: sunshine - Failed
Attempt 27: chocolate - Failed
Attempt 28: password1 - Failed
Attempt 29: soccer - Failed
Attempt 30: anthony - Failed
Attempt 31: friends - Failed
Attempt 32: butterfly - Failed
Attempt 33: purple - Failed
Attempt 34: angel - Failed
Attempt 35: jordan - Failed
```

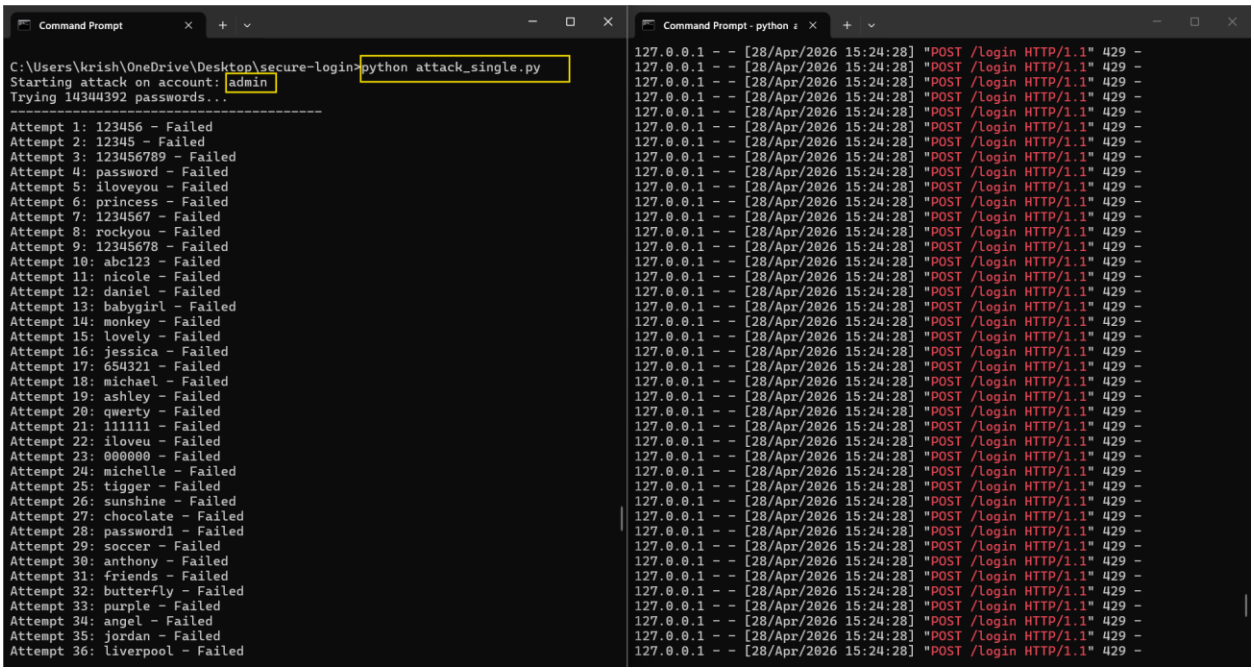
```
C:\Users\krish\OneDrive\Desktop\secure-login>python app.py
C:\Users\krish\AppData\Roaming\Python\Python311\site-packages\flask_limiter\
_extension.py:364: UserWarning: Using the in-memory storage for tracking rat
e limits as no storage was explicitly specified. This is not recommended for
 production use. See: https://flask-limiter.readthedocs.io#configuring-a-sto
rage-backend for documentation about configuring the storage backend.
  warnings.warn(
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deploym
ent. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
C:\Users\krish\AppData\Roaming\Python\Python311\site-packages\flask_limiter\
_extension.py:364: UserWarning: Using the in-memory storage for tracking rat
e limits as no storage was explicitly specified. This is not recommended for
 production use. See: https://flask-limiter.readthedocs.io#configuring-a-sto
rage-backend for documentation about configuring the storage backend.
  warnings.warn(
 * Debugger is active!
 * Debugger PIN: 406-883-278
127.0.0.1 - - [28/Apr/2026 15:18:02] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 15:18:02] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 429 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 429 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 429 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 429 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 429 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 429 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 429 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 429 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 429 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 429 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 429 -
127.0.0.1 - - [28/Apr/2026 15:18:03] "POST /login HTTP/1.1" 429 -
```

```
Attempt 165: sweetie - Failed
Attempt 166: 222222 - Failed
Attempt 167: beauty - Failed
Attempt 168: 987654 - Failed
Attempt 169: victoria - Failed
Attempt 170: honey - Failed
Attempt 171: 00000 - Failed
Attempt 172: fernando - Failed
Attempt 173: pokemon - Failed
Attempt 174: maggie - Failed
Attempt 175: corazon - Failed
Attempt 176: chicken - Failed
Attempt 177: pepper - Failed
Attempt 178: cristina - Failed
Attempt 179: rainbow - Failed
Attempt 180: kisses - Failed
Attempt 181: manuel - Failed
Attempt 182: myspace - Failed
Attempt 183: rebelde - Failed
Attempt 184: angel1 - Failed
Attempt 185: ricardo - Failed
Attempt 186: babygurl - Failed
Attempt 187: heaven - Failed
Attempt 188: 55555 - Failed
Attempt 189: baseball - Failed
Attempt 190: martin - Failed
Attempt 191: greenday - Failed
Attempt 192: november - Failed
Attempt 193: alyssa - Failed
Attempt 194: madison - Failed
Attempt 195: mother - Failed
Attempt 196: 123321 - Failed
Attempt 197: 123abc - Failed
Attempt 198: mahalkita - Failed
Attempt 199: batman - Failed
Attempt 200: september - Failed
Password not found for test

--------------------------------------
Attack complete! 0 accounts compromised:
```

Single account

With rate limiting enforced on the single-account attack, every subsequent request beyond the threshold is rejected with a **429 (Too Many Requests)** response, preventing any successful authentication. The attack script exhausts all 200 candidate passwords without retrieving a valid credential, demonstrating the effectiveness of rate limiting as a mitigation against targeted brute-force attempts.



```
Attempt 163: peanut - Failed
Attempt 164: blink182 - Failed
Attempt 165: sweetie - Failed
Attempt 166: 222222 - Failed
Attempt 167: beauty - Failed
Attempt 168: 987654 - Failed
Attempt 169: victoria - Failed
Attempt 170: honey - Failed
Attempt 171: 00000 - Failed
Attempt 172: fernando - Failed
Attempt 173: pokemon - Failed
Attempt 174: maggie - Failed
Attempt 175: corazon - Failed
Attempt 176: chicken - Failed
Attempt 177: pepper - Failed
Attempt 178: cristina - Failed
Attempt 179: rainbow - Failed
Attempt 180: kisses - Failed
Attempt 181: manuel - Failed
Attempt 182: myspace - Failed
Attempt 183: rebelde - Failed
Attempt 184: angel1 - Failed
Attempt 185: ricardo - Failed
Attempt 186: babygurl - Failed
Attempt 187: heaven - Failed
Attempt 188: 55555 - Failed
Attempt 189: baseball - Failed
Attempt 190: martin - Failed
Attempt 191: greenday - Failed
Attempt 192: november - Failed
Attempt 193: alyssa - Failed
Attempt 194: madison - Failed
Attempt 195: mother - Failed
Attempt 196: 123321 - Failed
Attempt 197: 123abc - Failed
Attempt 198: mahalkita - Failed
Attempt 199: batman - Failed
Attempt 200: september - Failed
Password not found in first 200 attempts

C:\Users\krish\OneDrive\Desktop\secure-login>
```

Conclusion

The credential stuffing attack was successfully simulated using rockyou.txt, compromising accounts that used weak passwords. After implementing rate limiting and account lockout, the attack was completely blocked with 0 accounts compromised, proving that these defenses are effective against automated login attacks.

3. Add TOTP-based 2FA using pyotp (Python) or speakeasy (Node.js). Show QR code generation and OTP verification. Demo a working 2FA login-QR scan on Google Authenticator, OTP entry, rejection of wrong OTP, and rejection of replay. Document the attack (how many accounts compromised before lockout). Submit before/after screenshots proving rate limiting blocks the attack.

https://www.npmjs.com/package/speakeasy

This task adds an extra layer of security to the login system using Time based One Time Passwords (TOTP). Even if an attacker knows the correct password, they cannot login without the 6 digit code generated by the authenticator app on the user's phone.

Tools and Technologies Used

- **Language:** Python
- **Framework:** Flask
- **2FA Library:** pyotp
- **QR Code Generation:** qrcode
- **Authenticator App:** Microsoft Authenticator

How TOTP Works

- When a user registers, a unique secret key is generated for them
- This secret is used to generate a QR code which the user scans with their authenticator app
- Every 30 seconds the authenticator app generates a new 6 digit code using this secret
- During login, after entering the correct password, the user must also enter this 6 digit code

The server verifies the code and only grants access if it matches

Installation:

The following packages were installed to implement TOTP based 2FA and QR code generation:

pip install pyotp qrcode[pil]

```
PS C:\Users\krish\OneDrive\Desktop\secure-login> pip install pyotp qrcode[pil]
Defaulting to user installation because normal site-packages is not writeable
Collecting pyotp
  Downloading pyotp-2.9.0-py3-none-any.whl (13 kB)
Collecting qrcode[pil]
  Downloading qrcode-8.2-py3-none-any.whl (45 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 46.0/46.0 kB 255.3 kB/s eta 0:00:00
Requirement already satisfied: colorama in c:\users\krish\appdata\roaming\python\python311\site-packages (from qrcode[pil]) (0.4.6)
Collecting pillow>=9.1.0 (from qrcode[pil])
  Downloading pillow-12.2.0-cp311-cp311-win_amd64.whl (7.1 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.1/7.1 MB 887.5 kB/s eta 0:00:00
Installing collected packages: qrcode, pyotp, pillow
Successfully installed pillow-12.2.0 pyotp-2.9.0 qrcode-8.2

[notice] A new release of pip is available: 23.1.2 -> 26.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\krish\OneDrive\Desktop\secure-login>
```

Step by Step Working

Step 1: Running the Application

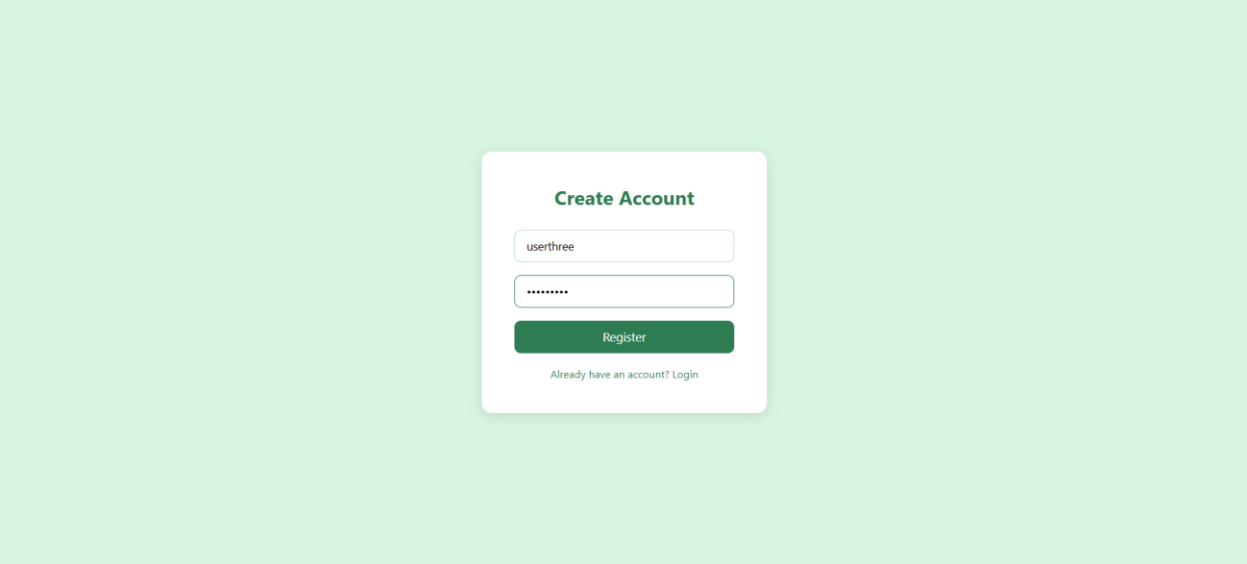
The application is started by running the following command in the terminal:

python app_qr.py

```
PS C:\Users\krish\OneDrive\Desktop\secure-login> python app_qr.py
C:\Users\krish\AppData\Roaming\Python\Python311\site-packages\flask_limiter\_extension.py:364: UserWarning: Using the in-memory storage for tracking rate li
mits as no storage was explicitly specified. This is not recommended for production use. See: https://flask-limiter.readthedocs.io#configuring-a-storage-bac
kend for documentation about configuring the storage backend.
  warnings.warn(
 * Serving Flask app 'app_qr'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
C:\Users\krish\AppData\Roaming\Python\Python311\site-packages\flask_limiter\_extension.py:364: UserWarning: Using the in-memory storage for tracking rate li
mits as no storage was explicitly specified. This is not recommended for production use. See: https://flask-limiter.readthedocs.io#configuring-a-storage-bac
kend for documentation about configuring the storage backend.
  warnings.warn(
 * Debugger is active!
 * Debugger PIN: 406-883-278
127.0.0.1 - - [29/Apr/2026 00:56:27] "GET /register HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 00:56:27] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [29/Apr/2026 00:56:39] "POST /register HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 00:56:39] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [29/Apr/2026 00:57:13] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 00:57:14] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [29/Apr/2026 00:57:22] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [29/Apr/2026 00:57:22] "GET /verify-otp HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 00:57:22] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [29/Apr/2026 00:57:38] "POST /verify-otp HTTP/1.1" 302 -
127.0.0.1 - - [29/Apr/2026 00:57:38] "GET /dashboard HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 00:57:38] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [29/Apr/2026 00:57:40] "GET /logout HTTP/1.1" 302 -
127.0.0.1 - - [29/Apr/2026 00:57:40] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 00:57:40] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [29/Apr/2026 00:57:50] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [29/Apr/2026 00:57:50] "GET /verify-otp HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 00:57:50] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [29/Apr/2026 00:57:57] "POST /verify-otp HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 00:57:57] "POST /predict HTTP/1.1" 404 -
127.0.0.1 - - [29/Apr/2026 00:59:21] "GET /register HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 00:59:21] "POST /predict HTTP/1.1" 404 -
```

Step 2: Registration

The user navigates to the registration page and enters a username and password.



Step 3: QR Code Generation

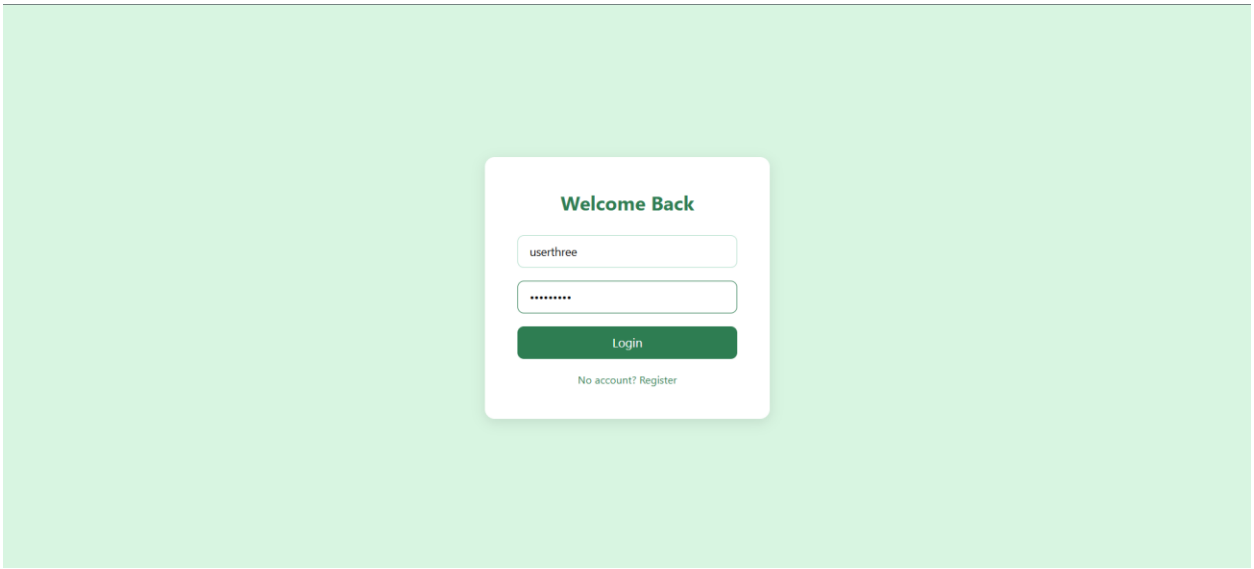
After registering, a unique QR code is generated for the user. The user must scan this QR code using Microsoft Authenticator or Google Authenticator to set up 2FA.

Step 4: Scanning QR Code on Google Authenticator

The user opens Google Authenticator on their phone and scans the QR code. The app then adds "userthree" and starts generating 6 digit codes that change every 30 seconds.

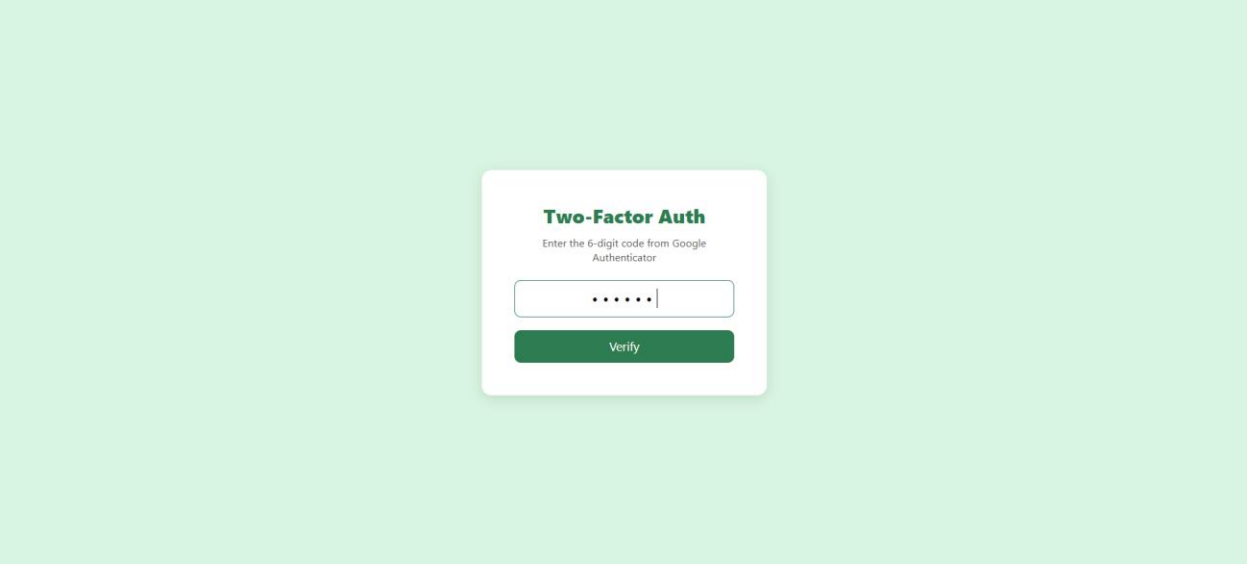
Step 5: Login with Password

The user navigates to the login page and enters their username and password. If the password is correct, they are redirected to the OTP verification page.



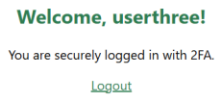
Step 6: OTP Verification

The user enters the 6 digit code shown in Microsoft Authenticator and clicks Verify.



Step 7: Login Successful

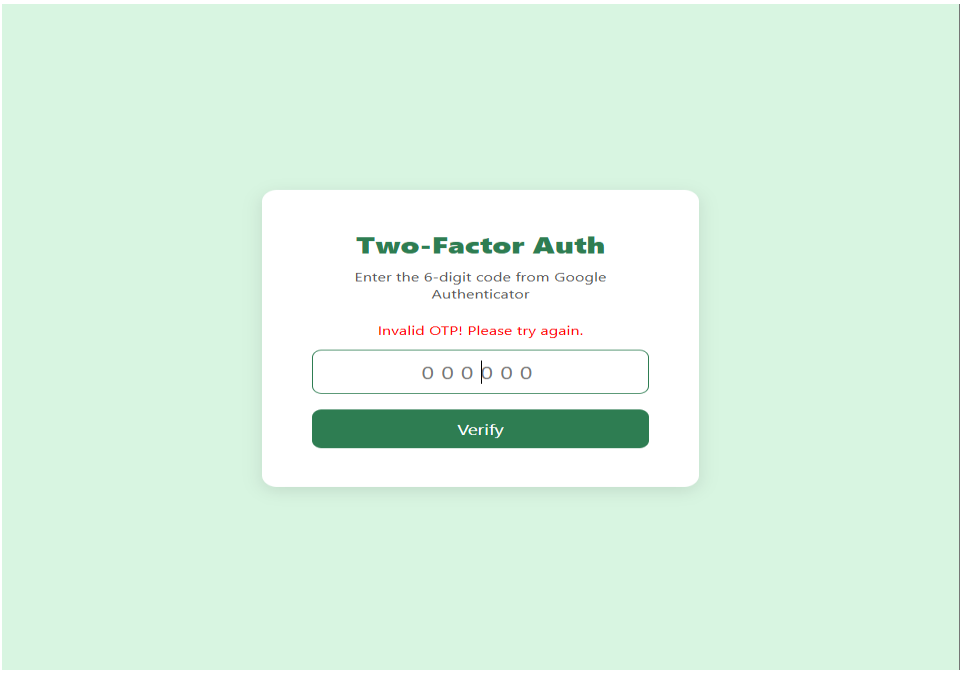
If the correct OTP is entered, the user is granted access and the welcome dashboard is shown.



<u>Wrong OTP :</u>

Step 8 :Wrong OTP Rejected

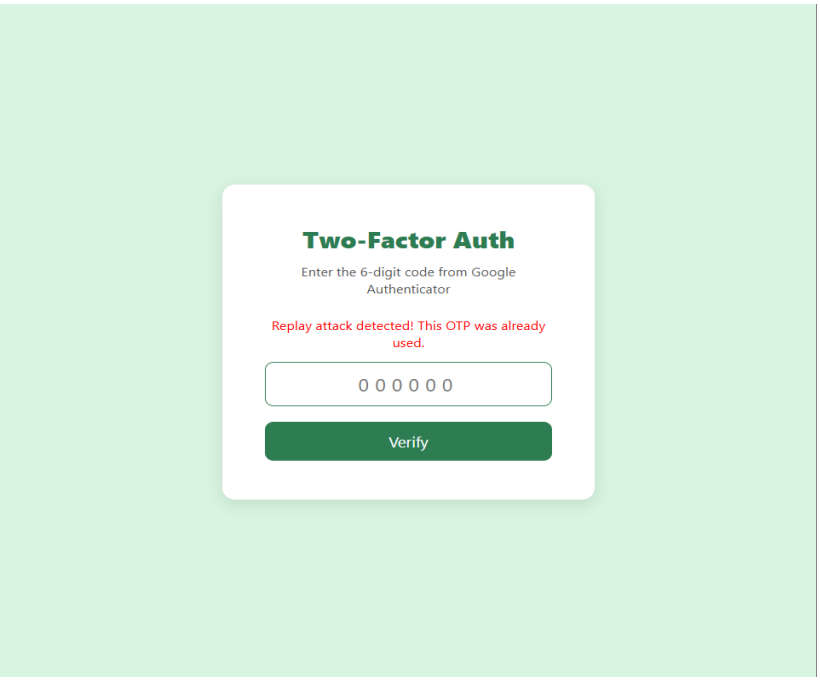
If the user enters an incorrect OTP, the system rejects it and shows an error message.



<u>Replay Attack:</u>

Step 9: Replay Attack Rejected

If the user tries to reuse an OTP that was already used, the system detects it as a replay attack and rejects it.



Conclusion

- Two factor authentication adds a second layer of security beyond just a password
- Even if an attacker knows the correct password they cannot login without the OTP
- OTPs expire every 30 seconds making them very hard to steal
- Replay attacks are blocked by tracking already used OTPs
- The system is now protected by both bcrypt password hashing and TOTP based 2FA